

Руководство пользователя: Система развертки цилиндрических поверхностей и распознавания текста (Dewarp & OCR Pipeline)

1. Описание программы

Программа представляет собой высокоточный гибридный конвейер компьютерного зрения. Ее главная задача — принимать фотографии цилиндрических объектов (бутылок, банок, контейнеров), сделанные в реальных условиях под любым углом, «разворачивать» (dewarping) их изогнутую поверхность в плоскую 2D-картинку без искажений, а затем распознавать на ней текст, логотипы, штрих-коды и QR-коды.

2. Системные требования

Поскольку программа использует методы глубокого обучения (Deep Learning) и сложную математическую оптимизацию, для комфортной работы требуется: * **ОС:** Windows 10/11, Linux (Ubuntu 20.04+) *

Процессор (CPU): Многоядерный процессор (Intel Core i5/i7 или AMD Ryzen 5/7) * **Видеокарта (GPU):** Рекомендуются NVIDIA с поддержкой CUDA (от 8 ГБ видеопамяти, например RTX 3060 или мощнее) для

работы нейросетей (YOLOv8-seg, UVDoc, PaddleOCR). * **Программное окружение:** Python 3.9+, установленные библиотеки (PyTorch, OpenCV, SciPy, NumPy).

3. Требования к входным изображениям (Фотографиям)

Для достижения наилучших результатов (высоких метрик MS-SSIM и низкого CER) соблюдайте следующие правила при съемке: 1.

Освещение: Старайтесь избегать сильных бликов (specular reflections) от стекла или глянцевого пластика, хотя программа имеет встроенный модуль коррекции освещения. 2. **Фокус и четкость:** Текст на этикетке должен быть в фокусе. Смазанные кадры (motion blur) сильно снижают качество распознавания (OCR). 3. **Полнота объекта:** Контейнер или хотя бы сама этикетка должны быть полностью видны в кадре. 4.

Разрешение: Рекомендуемое разрешение входных файлов от 1080p.

4. Как работает программа (Процесс обработки)

Когда вы загружаете фотографию в программу, она автоматически проходит 5 стадий обработки. Процесс полностью скрыт «под капотом», но вот что происходит:

- **Стадия 1: Локализация и подготовка.** Нейросеть (YOLOv8-seg/U2-Net) находит бутылку на фоне стола или комнаты, вырезает ее и убирает блики с помощью алгоритмов Retinex. Изображение переводится в бинарный (черно-белый) формат.
- **Стадия 2: Поиск геометрии.** Программа ищет строки текста и графические линии. По тому, как изогнуты строки, математика высчитывает, под каким углом была камера и какова кривизна бутылки.
- **Стадия 3: 3D-моделирование сетки.** Создается виртуальная 3D-модель (цилиндр). Программа оптимизирует параметры так, чтобы все кривые строки текста на модели стали идеально ровными горизонтальными линиями.
- **Стадия 4: Обратная деформация (Dewarping).** Изогнутое изображение натягивается на плоскую сетку (Thin-Plate Splines) и пересчитывается с использованием алгоритма Ланцоша (Lanczos), чтобы сохранить резкость мелкого шрифта.
- **Стадия 5: Распознавание (OCR).** Полученная плоская картинка отправляется в модуль распознавания. Весь текст переводится в текстовый файл, а выровненные QR-коды успешно считываются.

5. Использование через командную строку (CLI)

Предполагается, что вы будете запускать программу через консоль.

Основной скрипт — `run_pipeline.py`.

Базовый запуск (обработка одной картинки)

```
python run_pipeline.py --input /path/to/bottle.jpg --output_dir /path/to/re
```

В результате в папке результатов появятся: плоская картинка (`dewarped_bottle.jpg`) и текстовый файл (`ocr_results.json`).

Пакетная обработка папки (Batch Mode)

Если у вас есть папка с сотнями фотографий товаров:

```
python run_pipeline.py --input_dir /data/bottles/ --output_dir /data/result
```

Флаг `--save_intermediate True` сохранит картинки на каждом этапе (маски, сетки деформации), что полезно для отладки.

Дополнительные параметры

- `--ocr_model` — выбор движка распознавания (`paddleocr`, `tesseract`, `deepseek-vl`).
- `--fast_mode` — отключает сложную математическую оптимизацию LM (Levenberg-Marquardt), заменяя ее на быструю нейросеть (UVDoc). Работает быстрее, но может быть чуть менее точным на сложных формах.

6. Формат выходных данных

После успешной обработки одной фотографии программа генерирует 2 файла: 1. `flattened_image.png` — идеальная прямоугольная развертка этикетки (высокого разрешения, без бликов). 2. `results.json` — структурированный файл с распознанными данными: `json {`
`"text_blocks": [ {"text": "INGREDIENTS: CARBONATED WATER...", "confidence":`

```
0.98}, {"text": "500 ML", "confidence": 0.99} ], "barcodes": [ {"type":  
"QR_CODE", "data": "https://example.com/promo"} ], "metrics": {  
"dewarping_error_mse": 0.012 } }
```

7. Возможные проблемы и пути решения

Симптом	Возможная причина	Решение
Программа не нашла бутылку	Сложный фон или бутылка сливается с окружением.	Попробуйте сфотографировать объект на более контрастном фоне. Убедитесь, что <code>--fast_mode</code> отключен.
Текст распознался как "каша"	Сетка на Стадии 3 построилась криво из-за того, что на бутылке мало строк текста.	Включите глубокую нейросетевую оценку (<code>--use_d2dewarp</code>). Программа будет опираться не только на строки, но и на вертикальные границы этикетки.
QR-код размазан и не читается	Билинейная интерполяция испортила резкость.	В настройках (Стадия 4) убедитесь, что используется алгоритм передискретизации Ланцоша (Lanczos resampling).
Ошибка Out of Memory (OOM) на GPU	Входное разрешение фото слишком высокое (например 4K+).	Программа автоматически сжимает фото для поиска сетки, но для OCR использует оригинал. Закройте другие программы или уменьшите фото до 1920x1080.

Примечание: Данный конвейер является сложной модульной системой. Если вам необходимо модифицировать логику работы с кривизной (например, для объектов формы "песочные часы" или конусов), вам

потребуется внести изменения в формулы математического солвера (Levenberg-Marquardt) на Стадии 3.